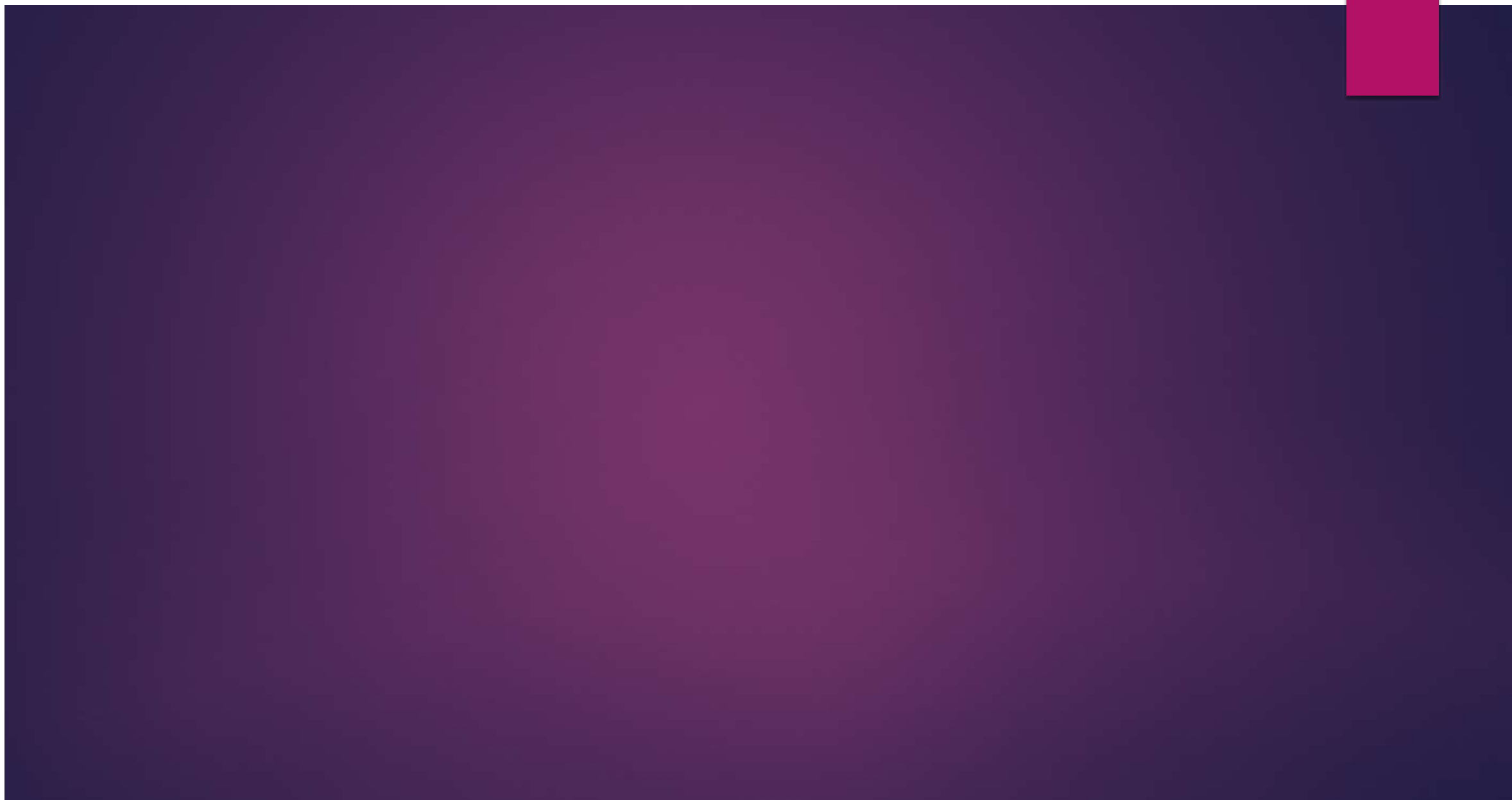




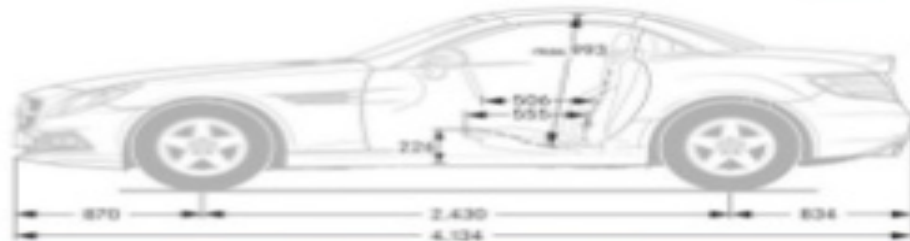
AGENDA

- Class Fundamentals
- Declaring Objects
- Example

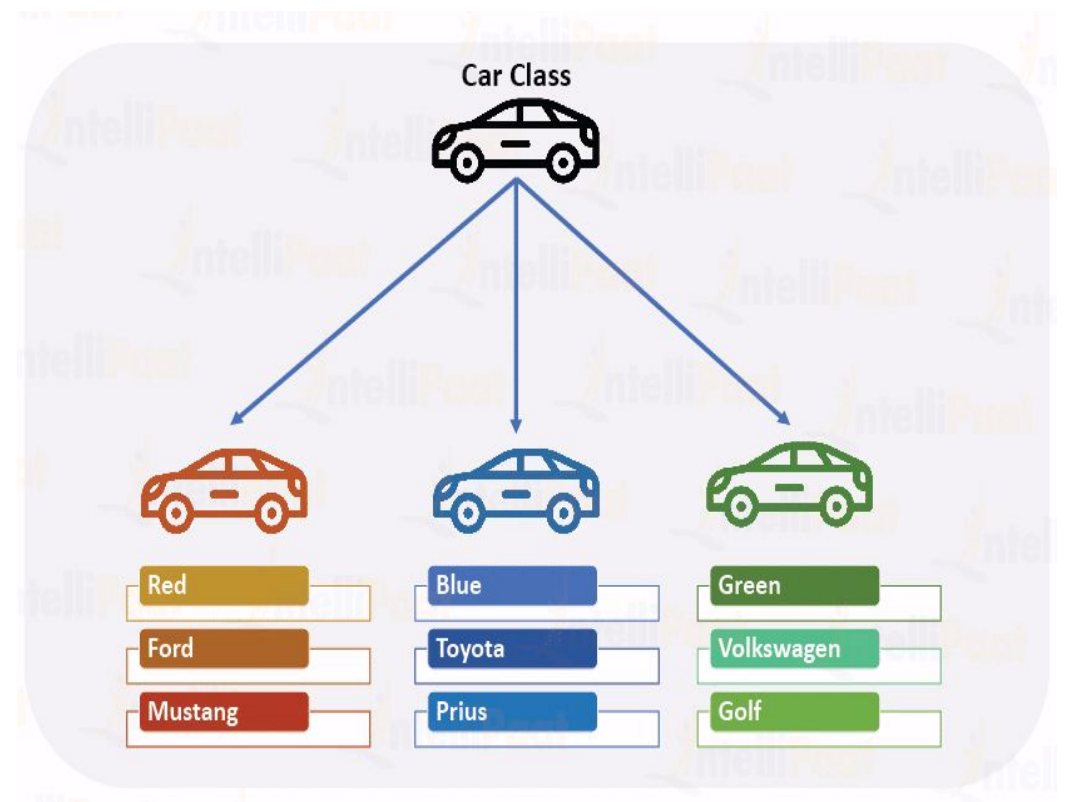
Class fundamentals

- ✓ A Class is a user-defined data type in Java that defines a template or blueprint for an object.
- ✓ We can create any number of variables (objects) of that data type.
- ✓ So, we can say that class is a template for an object and an object is an instance of a class.
- ✓ Most of the times, the terms object and instance are used interchangeably.

Class fundamentals



Think of the class like the Mercedes blue-print and the object like your brand new Mercedes.



Class fundamentals



Classes & Objects in Java

Class

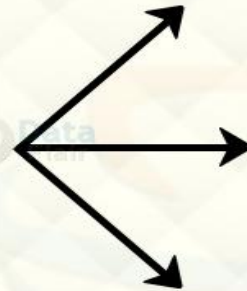
Car

**Data
Members**

model
color
brand

Methods

speed()
size()



model - Ertiga
color - Mehroon
brand - Maruti



model - XUV500
color - Black
brand - Mahindra



model - Swift
color - Red
brand - Maruti

Class fundamentals- declaring class

□ A class contains data (member or instance variables) and the code (member methods) that operate on the data.

□ The general form can be given as –

```
class classname
{
    type var1;
    type var2;
    .....
    type method1(para_list)
    {
        //body of method1
    }
    type method2(para_list)
    {
        //body of method2
    }
    .....
}
```

□ Here, classname is any valid name given to the class.

□ Variables declared within a class are called as instance variables because every instance (or object) of a class contains its own copy of these variables.

□ The code is contained within methods.

□ Methods and instance variables collectively called as members of the class

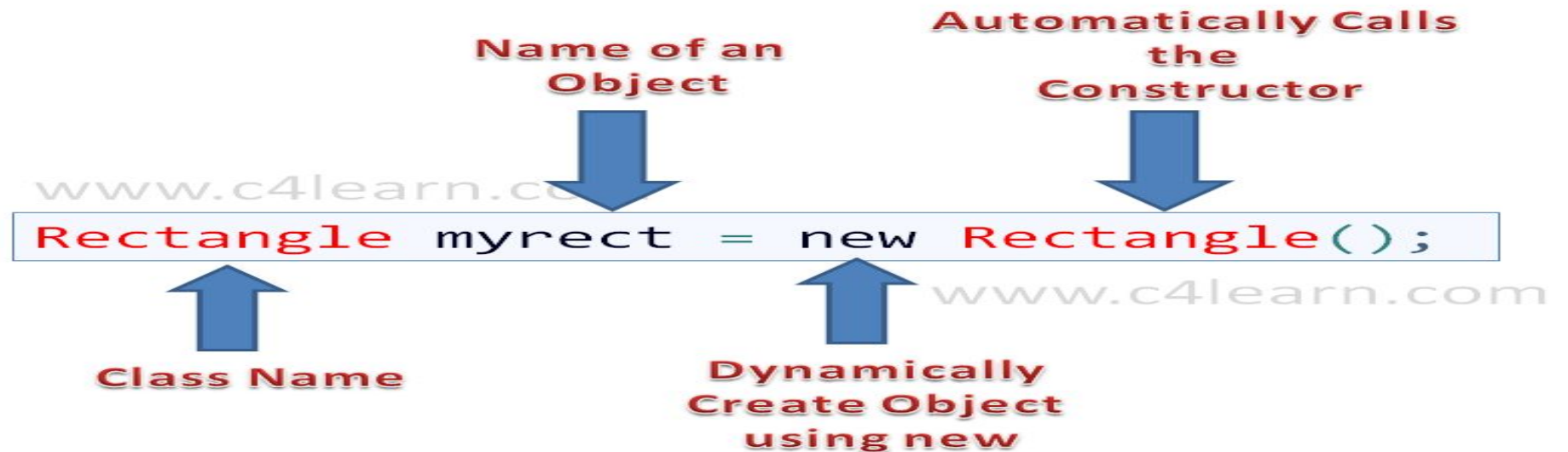
□ The instance variables are acted upon and accessed only the methods defined for that class

Class fundamentals- declaring object

- Creating a class means having a user-defined data type.
- To have a variable of this new data type, we should create an object
- Objects are created dynamically using the new operator
- Syntax: `<classname> <objectname>=new <classname>();`

declaring objects

- Creating a class means having a user-defined data type.
- To have a variable of this new data type, we should create an object
- Objects are created dynamically using the new operator
- Syntax: <classname> <objectname>=new <classname>();



declaring objects using new operator

- The new operator dynamically allocates the memory for an object
- The statement `Box mybox= new Box();` can be viewed as 2 statements as follows

`Box mybox;` **// declare a reference to object of the class Box and initialized to null**

`mybox= new Box();` **// allocate memory for Box object and assigns the reference to mybox**

declaring objects using new operator

Statement

Box mybox ;

Effect

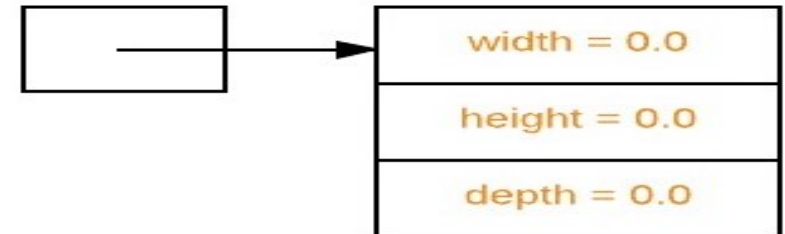
mybox



null

mybox = new Box() ;

mybox



Box object

Declaration, Instantiation and Initialization of an object of type Box

Assigning object reference variables

□ Object reference variables act differently when an assignment takes place.

□ For Example:

```
Box b1=new Box();
```

Is b2 assigned a copy of the object referred by b1, i.e., b1 and b2 refer to separate and distinct objects

```
Box b2=b1;
```

□ After this fragment executes, b1 and b2 will both refer to the same object.

□ The assignment of b1 to b2 did not allocate any memory or copy any part of the original object.

□ It simply makes b2 refer to the same object as does b1.

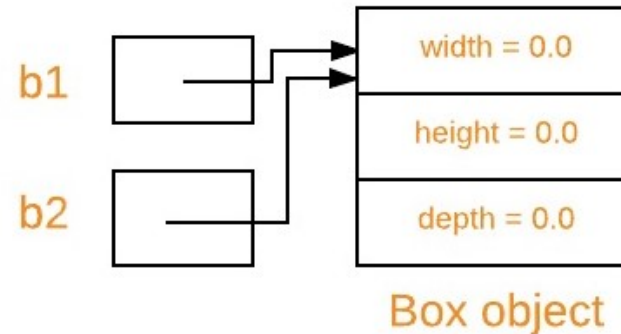
□ Thus, any changes made to the object through b2 will affect the object to which b1 is referring.

Assigning object reference variables

Statements

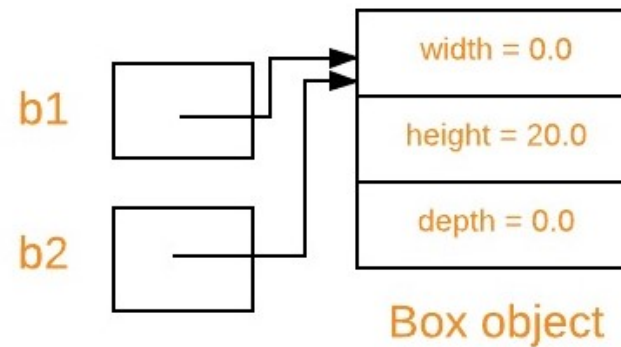
```
Box b1 = new Box();  
Box b2 = b1;
```

Effect



Box object

```
b2.height = 20 ;
```



Box object

Note: When we are assigning one object variable to another object variable, we are not creating copy of the object but making copy of the reference

Assigning object reference variables-example

```
class Box
{
    int width;
    int height;
    int depth;
}

public class Main
{
    public static void main(String args[])
    {
        Box myBox1 = new Box();
        Box myBox2 = myBox1;
        myBox1.width = 10;
        myBox2.width = 20;
        System.out.println("myBox1.width:" + myBox1.width);
        System.out.println("myBox2.width:" + myBox2.width);
    }
}
```

OUTPUT:

```
myBox1.width:20
myBox2.width:20
```

Introducing methods

- Classes usually consist of two things: instance variables and methods.
- Instance variables are the data part of a class, while the methods defines the behaviours of a class.
- Most of the time, methods are used to access the instance variables defined by the class

□ **Syntax:** This is the general form of a method:

```
type name(parameter-list)
{
    // body of method
}
```

- **type** specifies the type of data returned by the method. If the method does not return a value, its return type must be **void**.
- The name of the method is specified by name.
- The parameter-list is a sequence of type and identifier pairs separated by commas.
- Parameters receives the value of the arguments passed to the method. If the method has no parameters, then the parameter list will be empty.

Introducing methods - example

```
class Box
{
    int width;
    int height;
    int depth;
    void calculateVolume()
    {
        System.out.print("Volume is ");
        System.out.println(width * height * depth);
    }
}

public class BoxDemo
{
    public static void main(String args[])
    {
        Box mybox1 = new Box();
        mybox1.width = 10;
        mybox1.height = 20;
        mybox1.depth = 15;
        mybox1.calculateVolume();
    }
}
```

Note: When an instance variable is accessed by code that is not part of the class in which instance variable is defined , then it must be accessed through an object and dot operator.

However, When an instance variable is accessed by code that is part of the class in which instance variable is defined , then it can be referred directly.

This is applied to methods also.

constructor

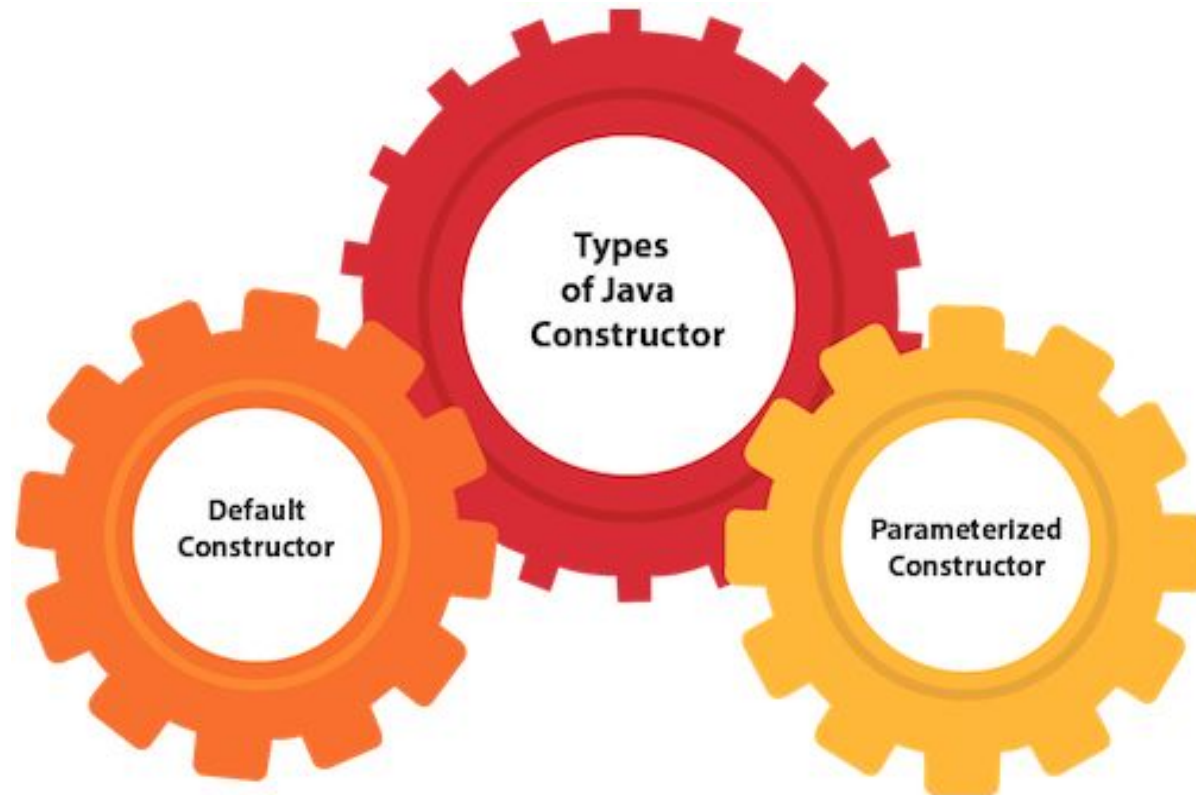
- In Java, a constructor is a block of codes similar to the method.
- It is called when an instance of the class is created.
- It is a special type of method which is used to initialize the object.
- Every time an object is created using the `new()` keyword, at least one constructor is called
- It calls a default constructor if there is no constructor available in the class.
- In such case, Java compiler provides a default constructor by default.
- There are two types of constructors in Java: Default Constructor/No-arg constructor, and parameterized constructor.

Rules for creating Java constructor

Rules defined for the constructor are:

- A Constructor name must be the same as its class name
- A Constructor must have no explicit return type
- A Constructor will be invoked automatically when an object is created (Explicitly Invoking not allowed)
- A Constructor can take zero or more arguments
- A Java constructor cannot be abstract, static, final, and synchronized

Types of constructor



Default constructor

- A "default constructor" that is automatically generated by the compiler if no constructors have been defined for the class.
- The default constructor executes an empty body.
- All the instance fields are initialized at their initial value of 0 (integer types), 0.0 (floating-point types), false (boolean type), or null (reference types).
- A programmer-defined constructor that takes no parameters is also called a default constructor

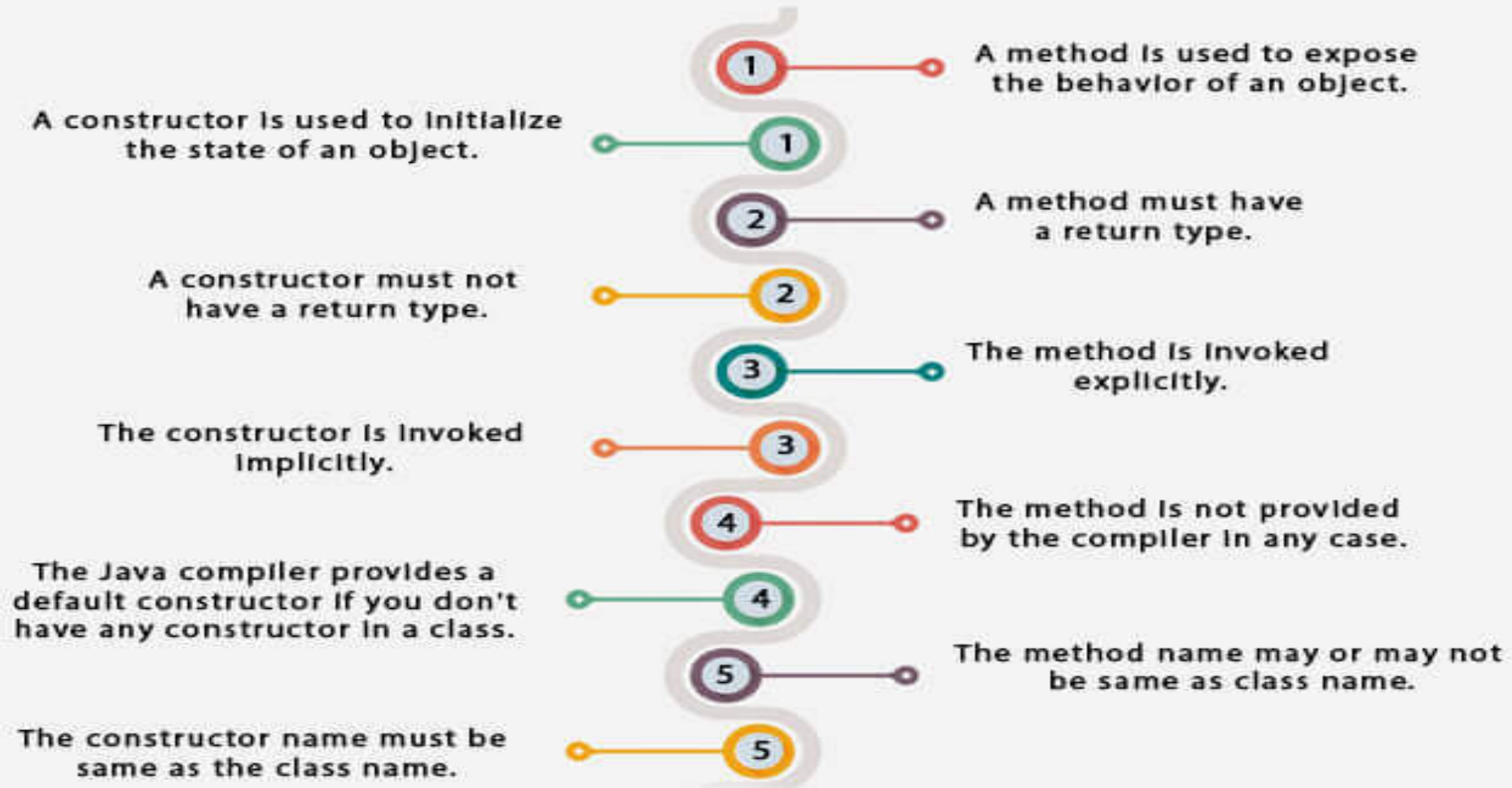
Default constructor

```
class classDemo{
    int num1;
    int num2;
    public void display(){
        System.out.println("num1 = "+ num1);
        System.out.println("num2 = "+ num2);
    }
}
class Main{
    public static void main(String args[]){
        classDemo cd1 = new classDemo();
        //create object of ClassDemo
        cd1.display();
    }
}
```

Compile

```
class classDemo{
    int num1;
    int num2;
    classDemo ()
    {
    }
    public void display(){
        System.out.println("num1 = "+ num1);
        System.out.println("num2 = "+ num2);
    }
}
class Main{
    public static void main(String args[]){
        classDemo cd1 = new classDemo();
        //create object of ClassDemo
        cd1.display();
    }
}
```

Difference between constructor and method in Java



Method overloading

- In Java it is possible to define two or more methods within the same class that share the same name, as long as their parameter declarations are different.
- When this is the case, the methods are said to be overloaded, and the process is referred to as method overloading.
- Method overloading is one of the ways that Java supports polymorphism.
- When an overloaded method is invoked, Java uses the type and/or number of arguments as its guide to determine which version of the overloaded method to actually call.
- Thus, overloaded methods must differ in the type and/or number of their parameters.
- While overloaded methods may have different return types, the return type alone is insufficient to distinguish two versions of a method.

Method overloading

- When an overloaded method is called, Java looks for a match between the arguments used to call the method and the method's parameters.
- However, this match need not always be exact. In some cases, Java's automatic type conversions can play a role in overload resolution.

Overloading constructors

- In addition to overloading normal methods, we can also overload constructors(Constructor is also a special method)
- In fact, for most real-world classes that we create, overloaded constructors will be the normal.
- Example:

Overloading constructors

class Box

{

double width;

double height;

double depth;

// constructor used when all dimensions specified

Box(double w, double h, double d)

{

width = w;

height = h;

depth = d;

}

// constructor used when no dimensions specified

Box()

{

width = -1; // use -1 to indicate

height = -1; // an uninitialized

depth = -1; // box

}

// constructor used when cube is created

Box(double len)

{

width = height = depth = len;

}

// compute and return volume

double volume()

{

return width * height * depth;

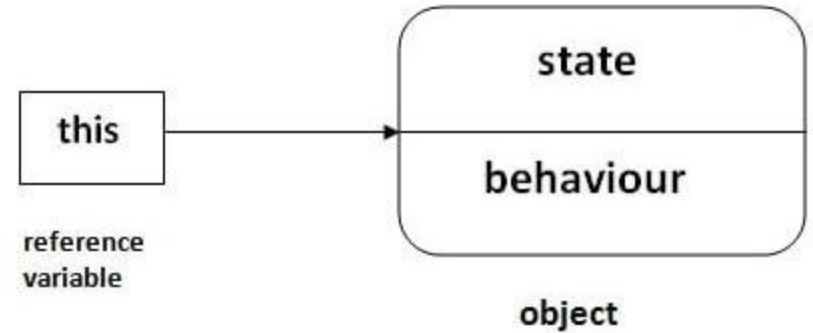
}

}

Overloading constructors

```
class OverloadCons
{
    public static void main(String args[])
    {
        // create boxes using the various constructors
        Box mybox1 = new Box(10, 20, 15);
        Box mybox2 = new Box();
        Box mycube = new Box(7);
        double vol;
        vol = mybox1.volume();
        System.out.println("Volume of mybox1 is " + vol);
        // get volume of second box
        vol = mybox2.volume();
        System.out.println("Volume of mybox2 is " + vol);
        // get volume of cube
        vol = mycube.volume();
        System.out.println("Volume of mycube is " + vol);
    }
}
```

this keyword



- Sometimes in a method will need to refer to the object that invoked it.
- To allow this, Java defines the **this** keyword.
- **this** can be used inside any method to refer to the *current* object
- The this keyword refers to the current object in a method or constructor.
- The most common use of the this keyword is to eliminate the confusion between class attributes and parameters with the same name (because a class attribute is shadowed by a method or constructor parameter).
- **this** keyword - To overcome shadowing or instance variable hiding.

this keyword- to refer current object in a method

```
class Student
{
    int rollno;
    String name;
    float fee;
    Student(int no,String n,float f)
    {
        rollno=no;
        name=n;
        fee=f;
    }
    void display()
    {
        System.out.println(rollno+" "+name+" "+fee);
    }
}
```

```
class TestThis1
{
    public static void main(String args[])
    {
        Student s1=new Student(111,"ankit",5000);
        Student s2=new Student(112,"sumit",6000);
        s1.display();
        s2.display();
    }
}
```

this keyword- instance variable hiding

- It is illegal in Java to declare two local variables with the same name inside the same or enclosing scopes.
- Interestingly, we can have local variables, including formal parameters to methods, which overlap with the names of the class' instance variables.
- However, when a local variable has the same name as an instance variable, the local variable ***hides*** the instance variable.
- To overcome this use this keyword- wrt instance variable.

this keyword- instance variable hiding

```
class Student
{
    int rollno;
    String name;
    float fee;
    Student(int rollno,String name,float fee)
    {
        this.rollno=rollno;
        this.name=name;
        this.fee=fee;
    }
    void display()
    {
        System.out.println(rollno+" "+name+" "+fee);
    }
}
```

```
class TestThis1
{
    public static void main(String args[])
    {
        Student s1=new Student(111,"ankit",5000);
        Student s2=new Student(112,"sumit",6000);
        s1.display();
        s2.display();
    }
}
```

static keyword

- Normally, a class member must be accessed only in conjunction with an object of its class.
- There will be times when we need to define a class member that will be used independently of any object of that class.
- However, it is possible to create a member that can be used by itself, without reference to a specific instance.
- To create such a member, precede its declaration with the keyword **static**.

static keyword



static keyword

- **When a member is declared static, it can be accessed before any objects of its class are created, and without reference to any object.**
- **We can declare both methods and variables to be static.**
- **The most common example of a static member is `main()`.**
- **`main()` is declared as static because it must be called before any objects exist.**
- **Instance variables declared as static are essentially, global variables.**
- **When objects of its class are declared, no copy of a static variable is made.**
- **Instead, all instances of the class share the same static variable.**
- **Methods declared as static have several restrictions:**
 - They can only call other **static** methods.
 - They must only access **static** data.
 - They cannot refer to **this** or **super** in any way

static keyword

// Demonstrate static variables, methods, and blocks.

```
class UseStatic
```

```
{
    static int a = 3;
    static int b;
    static void meth(int x)
    {
        System.out.println("x = " + x);
        System.out.println("a = " + a);
        System.out.println("b = " + b);
    }
    static
    {
        System.out.println("Static block initialized.");
        b = a * 4;
    }
    public static void main(String args[])
    {
        meth(42);
    }
}
```

static keyword

- Outside of the class in which they are defined, **static** methods and variables can be used independently of any object.
- To do so, we need only specify the name of their class followed by the dot operator.
- For example, if you wish to call a **static** method from outside its class, we can do so using the following general form:

classname.method()

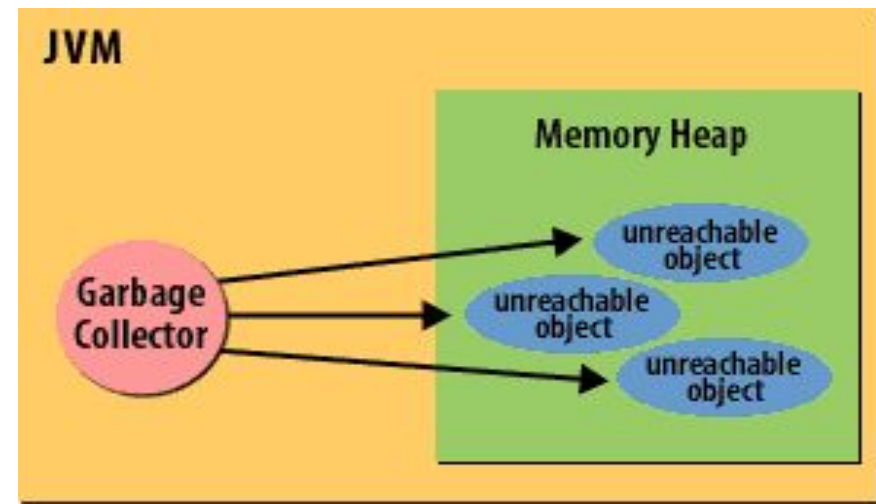
- Here, *classname* is the name of the class in which the **static** method is declared.
- A **static** variable can be accessed in the same way—by use of the dot operator on the name of the class.
- This is how Java implements a controlled version of global methods and global variables.

static keyword

```
class StaticDemo
{
    static int a = 42;
    static int b = 99;
    static void callme()
    {
        System.out.println("a = " + a);
    }
}
class StaticByName
{
    public static void main(String args[])
    {
        StaticDemo.callme();
        System.out.println("b = " + StaticDemo.b);
    }
}
```

Garbage Collection

- Since objects are dynamically allocated by using the **new** operator by programmer, need to analyze how such objects are destroyed and their memory released for later reallocation.
- Java takes a different approach; it handles deallocation for the programmers automatically.
- The technique that accomplishes this task is called ***garbage collection***.



Garbage Collection

It works like this: when no references to an object exist, that object is assumed to be no longer needed, and the memory occupied by the object can be reclaimed.

There is no explicit need to destroy objects as in C++(**use delete operator**).

Garbage collection only occurs sporadically (occasionally) during the execution of the program. It will not occur simply because one or more objects exist that are no longer used.

Types of Garbage Collector in Java

Serial Garbage Collector 01

Parallel Garbage Collector 02

CMS Garbage Collector 03

G1 Garbage Collector 04

Objects as parameters

- Till now, we have only been using simple datatypes like int, double as parameters to methods.
- However, it is also possible to pass objects to methods.
- When we pass an object to a method, the objects are passed by what call-by-reference.
- Thus, when we pass the reference to a method, the **parameter(in method definition)** that receives it will refer to the same object as that referred to by the **argument(method call)**.

Objects as parameters

```
class Test
{
    int a,b;
    Test(int i,int j)
    {
        a=i;
        b=j;
    }
    boolean equals(Test o)
    {
        if(o.a == a && o.b == b) //Here o refers to the same object referred by t2
            return true; //a and b refers to the invoking object
        else
            return false;
    }
}
```

```
public class OverloadCons
{
    public static void main(String args[])
    {
        Test t1=new Test(100,22);
        Test t2=new Test(100,22);
        Test t3=new Test(10,22);
        System.out.println("t1==t2::"+t1.equals(t2));
        System.out.println("t1==t3::"+t1.equals(t3));
    }
}
```

OUTPUT:

```
t1 == t2:: true
t1== t3:: false
```


Returning Objects

- Till now, we have only been using methods returning simple datatypes like int, double
- However, it is also possible to return objects by methods.
- A method can return any type of data, including class types that user creates.

Returning Objects

```
class Test
{
    int a;
    Test(int i)
    {
        a = i;
    }
    Test incrByTen()
    {
        Test temp = new Test(a+10);
        return temp;
    }
}
```

```
class RetOb
{
    public static void main(String args[])
    {
        Test ob1 = new Test(2);
        Test ob2;
        ob2 = ob1.incrByTen();
        System.out.println("ob1.a: " + ob1.a);
        System.out.println("ob2.a: " + ob2.a);
        ob2 = ob2.incrByTen();
        System.out.println("ob2.a after second increase: " + ob2.a);
    }
}
```

OUTPUT:
ob1.a:2
ob2.a:12
ob2.a after second increase:22

Argument passing

- In general, there are two ways that a computer language can pass an argument to a subroutine.
- The first way is *call-by-value*. This approach copies the *value* of an argument into the formal parameter of the subroutine. Therefore, changes made to the parameter of the subroutine have no effect on the argument.
- The second way an argument can be passed is *call-by-reference*. In this approach, a reference to an argument (not the value of the argument) is passed to the parameter.
- Inside the subroutine, this reference is used to access the actual argument specified in the call. This means that changes made to the parameter will affect the argument used to call the subroutine.

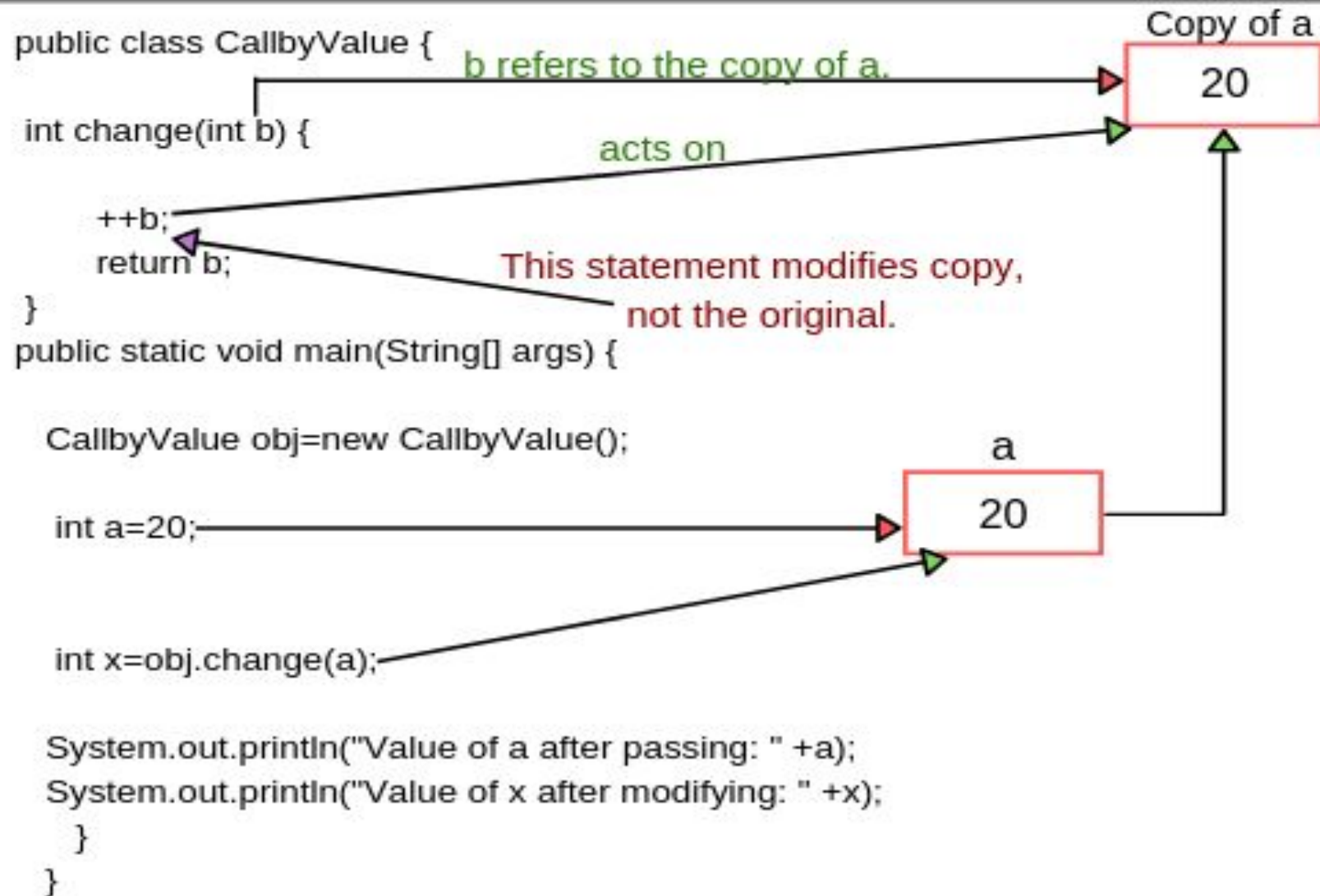


Fig: Call by Value mechanism

Argument passing-call by value

// Primitive types are passed by value.

```
class Test
{
    void meth(int i, int j)
    {
        i *= 2;
        j /= 2;
    }
}
```

```
class CallByValue
{
    public static void main(String args[])
    {
        Test ob = new Test();
        int a = 15, b = 20;
        System.out.println("a and b before call: " + a + " " + b);
        ob.meth(a, b);
        System.out.println("a and b after call: " + a + " " + b);
    }
}
```

OUTPUT:

a and b before call: 15 20

a and b after call: 15 20

Argument passing

- ❑ In Java, when we pass a primitive type to a method, it is passed by value.
- ❑ Thus, what occurs to the parameter that receives the argument has no effect outside the method.
- ❑ When we pass an object to a method, the situation changes dramatically, because objects are passed by what is effectively call-by-reference.
- ❑ Thus, when we pass this reference to a method, the parameter that receives it will refer to the same object as that referred to by the argument.
- ❑ This effectively means that objects are passed to methods by use of call-by-reference. Changes to the object inside the method do affect the object used as an argument.

Argument passing-call by reference

// Objects are passed by reference.

```
class Test
{
    int a, b;
    Test(int i, int j)
    {
        a = i;
        b = j;
    }
    // pass an object
    void meth(Test o)
    {
        o.a *= 2;
        o.b /= 2;
    }
}
```

```
class CallByRef
{
    public static void main(String args[])
    {
        Test ob = new Test(15, 20);
        System.out.println("ob.a and ob.b before call: " + ob.a + " " + ob.b);
        ob.meth(ob);
        System.out.println("ob.a and ob.b after call: " + ob.a + " " + ob.b);
    }
}
```

OUTPUT:

```
ob.a and ob.b before call: 15 20
ob.a and ob.b after call: 30 10
```

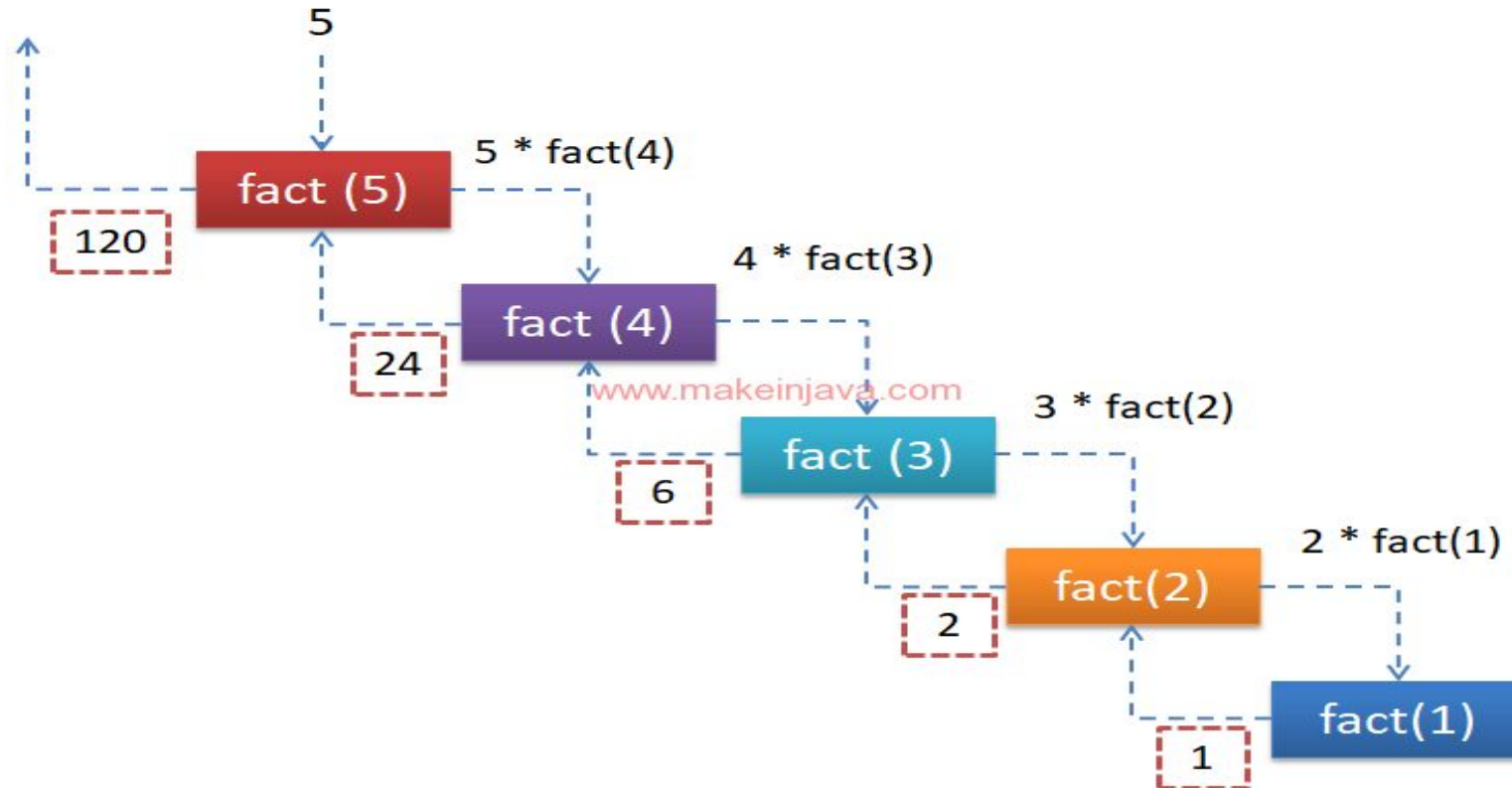
recursion

- Java supports *recursion*.
- Recursion is the process of defining something in terms of itself.
- As it relates to Java programming, recursion is the attribute that allows a method to call itself.
- A method that calls itself is said to be *recursive*.
- The classic example of recursion is the computation of the factorial of a number.
- The factorial of a number N is the product of all the whole numbers between 1 and N .
- For example, 3 factorial is $1 \times 2 \times 3$, or 6.
- Here is how a factorial can be computed by use of the recursive method and following recursive definition:

factorial(n)= n *factorial($n-1$) when $n > 1$ → **General Case**

factorial(1)=1 when $n=1$ → **Base Case**

recursion



recursion

```
// A simple example of recursion.
class Factorial
{
    // this is a recursive method
    int fact(int n)
    {
        int result;
        if(n==1)
            return 1;
        result = fact(n-1) * n;
        return result;
    }
}
```

```
class Recursion
{
    public static void main(String args[])
    {
        Factorial f = new Factorial();
        System.out.println("Factorial of 3 is " + f.fact(3));
        System.out.println("Factorial of 4 is " + f.fact(4));
        System.out.println("Factorial of 5 is " + f.fact(5));
    }
}
```

OUTPUT:

```
Factorial of 3 is 6
Factorial of 4 is 24
Factorial of 5 is 120
```

Access control specifiers

- We know that encapsulation links data with the code that manipulates it
- Encapsulation provides another important attribute: *access control*.
- Through encapsulation, we can control what parts of a program can access the members of a class.
- By controlling access, we can prevent misuse. For example, allowing access to data only through a well defined set of methods, we can prevent the misuse of that data.
- Thus, when correctly implemented, a class creates a “*black box*” which may be used, but the inner workings of which are not open to tampering
- Java’s access specifiers are *public, private, and protected*.
- Java also defines a *default access level*

Access control specifiers

- **Protected** applies only when inheritance is involved
- When a member of a class is modified by the **public** specifier, then that member can be accessed by any other code.
- When a member of a class is specified as **private**, then that member can only be accessed by other members of its class.
- When no access specifier is used, then by default the member of a class is public within its own package(A package is grouping of classes), but cannot be accessed outside of its package.

Access control specifiers

Access Specifiers in Java

		public	private	protected	default
Same Package	Class	YES	YES	YES	YES
	Sub class	YES	NO	YES	YES
	Non sub class	YES	NO	YES	YES
Different Package	Sub class	YES	NO	YES	NO
	Non sub class	YES	NO	NO	NO

Access control specifiers

□ An access specifier precedes the rest of a member's type specification. That is, it must begin a member's declaration statement.

□ Here is an example:

```
public int i;  
private double j;  
private int myMethod(int a, char b) { // ...
```

□ Program Example:

“

Thank You!

”

ANY QUESTIONS?